

CampusEats SOAP Partner Integration

Assignment 3 Technical Report & Design Brief

CS 543 Web Services — Assignment 3

Team ID: Team 10

Members:

- Nandini Kanaujiya (Roll: 20251651061)
- Radhika Verma (Roll: 20251651075)
- Alka Jha (Roll: 20251651013)
- Alok Mishra (Roll: 20251651014)

1. Context & SOAP Justification

CampusEats requires integration with an external payment gateway, **SecurePay**, to authorize and capture credit card payments for food orders. The operation selected for integration is **processPayment**. While the internal services of CampusEats are built on a modern, lightweight REST/JSON architecture, the connection to the payment partner must use SOAP for several critical engineering reasons:

1. **Strict Strongly-Typed Contract:** Financial services demand zero structural errors in exchange payloads. The WSDL file enforces strict XML schema validation at compile-time and runtime, preventing payload mismatches or missing fields that are common with loose JSON schemas.
2. **Message-Level Security:** Unlike REST, which relies solely on transport security (HTTPS/TLS), SOAP natively supports WS-Security. This allows signing and encrypting specific parts of the XML payload (e.g., credit card number) so they remain secure and tamper-proof even if processed through multiple intermediate proxies or hops.
3. **Transactional Integrity:** SOAP supports native transaction standards (WS-AtomicTransaction) which ensure that order processing and charge capture are processed atomically. This guarantees that either the transaction completes successfully or rolls back completely, preventing scenarios where a card is charged but the order fails to lodge internally.

2. HTTP POST Binding Details

The SOAP request is carried over HTTP using a POST method. The following block details the raw HTTP request structure containing the SOAP envelope payloads, headers, and endpoint URL.

POST /soap/payment HTTP/1.1

Host: api.securepay.com

Content-Type: text/xml; charset=utf-8

Content-Length: 642

SOAPAction: "http://securepay.com/payment/processPayment"

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"  
    xmlns:tns="http://securepay.com/payment/types/">
```

```
    <soapenv:Header>
```

```
        <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssec  
        urity-secect-1.0.xsd">
```

```
            <wsse:UsernameToken>
```

```
                <wsse:Username>campuseats_partner_10</wsse:Username>
```

```
                <wsse:Password>SecureTokenPass2026</wsse:Password>
```

```
            </wsse:UsernameToken>
```

```
        </wsse:Security>
```

```
    </soapenv:Header>
```

```
    <soapenv:Body>
```

```
        <tns:processPaymentRequest>
```

```
            <tns:cardHolderName>Nandini Kanaujiya</tns:cardHolderName>
```

```
            <tns:cardNumber>4111222233334444</tns:cardNumber>
```

```
            <tns:cvv>123</tns:cvv>
```

```
            <tns:expiryDate>12/28</tns:expiryDate>
```

```
            <tns:amount>450.50</tns:amount>
```

```
            <tns:currency>INR</tns:currency>
```

```
        </tns:processPaymentRequest>
```

```
    </soapenv:Body>
```

```
</soapenv:Envelope>
```

3. Modern Discovery & Registry

In modern microservice architectures, active discovery via UDDI servers is obsolete. Instead, services are discovered using a developer portal or central catalog (e.g., Kong Developer Portal, MuleSoft Exchange, or a private Spotify Backstage instance).

The catalog entry stores business context, technical endpoints, and the link to the WSDL contract. The WSDL file represents the "tModel" pointer that acts as the technical footprint for consumer binding.

```
{
  "registry": "CampusEats Enterprise API Registry",
  "business": {
    "name": "SecurePay Inc.",
    "contact": "support@securepay.com",
    "description": "Enterprise credit card processing provider"
  },
  "service": {
    "name": "SecurePay Payment Service",
    "description": "Real-time payment authorization and capture",
    "category": "Financial Services"
  },
  "endpoint": {
    "url": "https://api.securepay.com/soap/payment",
    "protocol": "SOAP 1.1 over HTTP",
    "wsdlUrl": "https://api.securepay.com/soap/payment?wsdl",
    "wsdlFingerprint": "tModel:securepay-payment-v1.1"
  }
}
```

4. Fault Mapping Scheme

To prevent leaking the partner's internal implementation details, namespaces, or stack traces (e.g. soapenv:Fault structures) to CampusEats clients, the payment adapter translates partner-specific errors into standard CampusEats REST errors (specified in Assignment 2).

Below is the translation table showing how the SOAP fault maps into the corresponding HTTP status and JSON response block.

SOAP Fault Field	Value / Code	REST Output Field	REST Value
faultcode	soapenv:Client	HTTP Status	400 Bad Request
errorCode	SP-402	code	payment_declined
errorMessage	Declined: Insufficient Funds	message	Payment failed: Insufficient account funds.

REST JSON Error Payload Returned to Client:

```
{
  "status": "error",
  "code": "payment_declined",
  "message": "Payment failed: Insufficient account funds."
}
```